

Data Preparation

Data upload

```
DROP TABLE IF EXISTS pizza_sales;
create table pizza_sales(
pizza_id int,
order_id int,
pizza_name_id text,
quantity int,
order_date text,
order_year int,
order_month text,
order_time time,
Hour_value int,
Day_part text,
unit_price double,
total_price double,
pizza_size text ,
pizza_category text,
pizza_ingredients text,
pizza_name text
);
select * from pizza_sales;

LOAD DATA INFILE 'Pizza_sales_updated.csv'
INTO TABLE pizza_sales
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(pizza_id, order_id, pizza_name_id, quantity, order_date, order_year, order_month,
order_time, Hour_value, Day_part, unit_price, total_price, pizza_size,
pizza_category, pizza_ingredients, pizza_name);

create table pizza_sales_replica like pizza_sales;

insert into pizza_sales_replica
select * from pizza_sales;
select * from pizza_sales_replica;
```

1. COLUMN data type change and removal.

```
select order_date, str_to_date(order_date,'%d/%m/%Y') as order_date_updated
from pizza_sales_replica;

alter table pizza_sales_replica
add column order_date_updated date after order_date;

update pizza_sales_replica
```

```
SET order_date_updated = str_to_date(order_date,'%d/%m/%Y');
```

```
ALTER TABLE pizza_sales_replica  
DROP COLUMN order_date;
```

```
ALTER TABLE pizza_sales_replica  
CHANGE COLUMN order_date_updated order_date DATE;
```

2. Find character position

```
select pizza_name_id,locate('_',pizza_name_id)  
from pizza_sales_replica;
```

3. Replacing a character with another value

```
select pizza_name_id,  
replace(pizza_name_id,right(pizza_name_id,1),if(right(pizza_name_id,1)='m','medium'  
,  
if(right(pizza_name_id,1)='l','large','small')))) as pizza_name_id_full  
from pizza_sales_replica;
```

4. Creating new modified column

```
update pizza_sales_replica  
set pizza_name_id_full =  
replace(pizza_name_id,right(pizza_name_id,1),if(right(pizza_name_id,1)='m','medium'  
,  
if(right(pizza_name_id,1)='l','large','small'))));
```

SQL Analysis Questions (Intermediate to Advanced)

♦ Time & Trend Analysis

1. What is the **monthly revenue trend** across years?

```
select order_year,extract(month from order_date) as month_number,  
sum(total_price) as monthly_revenue  
from pizza_sales_replica  
group by 1,2  
;
```

	order_year	month_number	monthly_revenue
▶	2015	1	69793.2999999999
	2015	2	65159.5999999992
	2015	3	70397.0999999998
	2015	4	68736.7999999987
	2015	5	71402.7499999988

2. What are the **peak hours** and **peak days** based on order volume and revenue?

```
select Hour_value as time_val_in_a_day, sum(quantity) as total_orders,
sum(total_price) as total_revenue
from pizza_sales_replica
group by 1
;
```

	time_val_in_a_day	total_orders	total_revenue
▶	9	4	83
	10	18	303.65
	11	2728	44935.7999999996
	12	6776	111877.8999999951
	13	6413	106065.6999999958

```
select dayofweek(order_date) as day_value, dayname(order_date) as Day_name,
sum(quantity) as total_orders,
sum(total_price) as total_revenue
from pizza_sales_replica
group by 1,2
;
```

	day_value	Day_name	total_orders	total_revenue
▶	1	Sunday	6035	99203.4999999967
	2	Monday	6485	107329.5499999958
	3	Tuesday	6895	114133.7999999958
	4	Wednesday	6946	114408.3999999946
	5	Thursday	7478	123528.4999999945

3. Which **daypart (Morning, Afternoon, Evening, Night)** drives the most revenue?

```
-- Which daypart (Morning, Afternoon, Evening, Night) drives the most revenue?
select Day_part, round(sum(total_price),0) as total_revenue
```

```

from pizza_sales_replica
group by 1
order by 2 desc;

```

	Day_part	total_revenue
►	Afternoon	400193
	Evening	306379
	Night	65966
	Morning	45322

4. Analyze the **seasonal performance** of pizzas.

-- Analyze the seasonal performance of pizzas for top 3 pizzas.

```

select *
from
(select *,
dense_rank() over(partition by season order by total_orders desc,total_revenue desc ) as
top_rank
from
(select pizza_name,
sum(quantity) as total_orders,
sum(total_price) as total_revenue,
season
from
(select pizza_name,
quantity, total_price,
case
    when extract(month from order_date) in (3,4,5) then 'spring'
    when extract(month from order_date) in (6,7,8) then 'summer'
    when extract(month from order_date) in (9,10,11) then 'fall'
    else 'winter'
end as season
from pizza_sales_replica) base
group by 1,4) final) ranked
where top_rank in (1,2,3)
;

```

	pizza_name	total_orders	total_revenue	season	top_rank
►	The Classic Deluxe Pizza	623	9712.5	fall	1
	The Hawaiian Pizza	611	8083.75	fall	2
	The Thai Chicken Pizza	602	11015.5	fall	3
	The Barbecue Chicken Pizza	671	11791.25	spring	1
	The Classic Deluxe Pizza	638	9954.5	spring	2

5. What is the **sales trend by pizza category over time**?

```
select extract(month from order_date) as Month_number, order_month, pizza_category,
sum(quantity) as total_orders, round(sum(total_price),0) as total_revenue
from pizza_sales_replica
group by 1,2,3;
```

	Month_number	order_month	pizza_category	total_orders	total_revenue
▶	1	January	Chicken	913	16189
	1	January	Classic	1257	18619
	1	January	Supreme	1044	17930
	1	January	Veggie	1018	17055
	2	February	Chicken	875	15468
	2	February	Classic	1178	17336
	2	February	Supreme	964	16719

◆ Product Performance & Profitability

6. Which are the **Top 10 pizzas by revenue and quantity**?

```
select pizza_name, sum(quantity) as total_quantity, sum(revenue) as total_revenue
from pizza_sales_replica
group by 1
order by total_quantity desc, total_revenue desc
limit 10;
```

	pizza_name	total_quantity	total_revenue
▶	The Classic Deluxe Pizza	2453	38180.5
	The Barbecue Chicken Pizza	2432	42768
	The Hawaiian Pizza	2422	32273.25
	The Pepperoni Pizza	2418	30161.75
	The Thai Chicken Pizza	2371	43434.25
	The California Chicken Pizza	2370	41409.5
	The Sicilian Pizza	1938	30940.5
	The Spicy Italian Pizza	1924	34831.25
	The Southwest Chicken Pizza	1917	34705.75
	The Big Meat Pizza	1914	22968

7. What is the **revenue per pizza category and size**?

```
select pizza_category, cast(sum(revenue) as decimal(10,0)) as total_revenue
from pizza_sales_replica
group by 1
order by total_revenue desc;
```

```
select pizza_size, cast(sum(revenue) as decimal(10,0)) as total_revenue
from pizza_sales_replica
group by 1
order by total_revenue desc;
```

pizza_category	total_revenue
Classic	220053
Supreme	208197
Chicken	195920
Veggie	193690

pizza_size	total_revenue
Large	375319
Medium	249382
Small	178076
XLarge	14076
XXLarge	1007

8. What is the **most profitable pizza** in terms of total revenue?

```
select pizza_name, cast(sum(revenue) as decimal(10,0)) as total_revenue
from pizza_sales_replica
group by 1
order by total_revenue desc
limit 1;
```

pizza_name	total_revenue
The Thai Chicken Pizza	43434

9. Calculate **average unit price by pizza size and category**.

```
select pizza_size, cast(avg(unit_price) as decimal(10,0)) as avg_unit_price
from pizza_sales_replica
group by 1
order by 2 desc;
```

```
select pizza_category, cast(avg(unit_price) as decimal(10,0)) as avg_unit_price
from pizza_sales_replica
group by 1
order by 2 desc;
```

pizza_size	avg_unit_price
XXLarge	36
XLarge	26
Large	20
Medium	16
Small	12

pizza_category	avg_unit_price
Chicken	18
Veggie	17
Supreme	17
Classic	15

10. Rank pizzas by their **contribution to total revenue (% share)**.

-- Rank pizzas by their contribution to total revenue (% share).

```
with table1 as(
select pizza_name, cast(sum(revenue) as decimal(10,0)) as total_revenue
from pizza_sales_replica
group by 1
),
```

```
table2 as (
select sum(total_revenue) as overall_revenue from table1
)
```

```
select t1.pizza_name, t1.total_revenue,
cast((t1.total_revenue/t2.overall_revenue)*100 as decimal(10,2)) as
contribution_percentage,
rank() over(order by contribution_percentage desc ) as Ranks
from table1 t1
cross join table2 t2
order by contribution_percentage desc;
```

pizza_name	total_revenue	contribution_percentage	Ranks
The Thai Chicken Pizza	43434	5.31	1
The Barbecue Chicken Pizza	42768	5.23	2
The California Chicken Pizza	41410	5.06	3
The Classic Deluxe Pizza	38181	4.67	4
The Spicy Italian Pizza	34831	4.26	5
The Southwest Chicken Pizza	34706	4.24	6
The Italian Supreme Pizza	33477	4.09	7
The Four Cheese Pizza	32266	3.95	8
The Hawaiian Pizza	32273	3.95	8
The Sicilian Pizza	30941	3.78	10

♦ Customer Behavior (Order Pattern Insight)

11. What is the **average number of pizzas per order**?

```
-- Step 1: Set the variable @a with the count of distinct order_id
select count(distinct order_id) into @a from pizza_sales_replica;
select sum(quantity)/@a as average_pizzas_per_order
from pizza_sales_replica;
```

average_pizzas_per_order
2.3220

12. Which pizzas are **frequently ordered together** (pair analysis)?

```
SELECT
    CONCAT(LEAST(p1.pizza_name, p2.pizza_name), '-', GREATEST(p1.pizza_name,
p2.pizza_name)) AS pizza_pair,
    COUNT(*) AS pair_count
FROM pizza_sales_replica p1
JOIN pizza_sales_replica p2
    ON p1.order_id = p2.order_id
    AND p1.pizza_name < p2.pizza_name -- Prevent duplicate pairs like (Cheese,
Pepperoni) and (Pepperoni, Cheese)
GROUP BY pizza_pair
order by pair_count desc
Limit 10;
```

pizza_pair	pair_count
The Hawaiian Pizza-The Thai Chicken Pizza	319
The Barbecue Chicken Pizza-The Pepperoni Pizza	308
The Pepperoni Pizza-The Thai Chicken Pizza	299
The Hawaiian Pizza-The Pepperoni Pizza	299
The Classic Deluxe Pizza-The Hawaiian Pizza	296

13. What are the **top 10 ingredients used in best-sellers**?

```
-- What are the top 10 ingredients used in best-sellers?
```

```
with tab1 as (
select row_number() over(partition by pizza_ingredients) as
rowNum,pizza_ingredients,
replace(replace(pizza_ingredients,'"','"),'?','') as pizzaIngredients
from pizza_sales_replica),
tab2 as (
select t1.rowNum,t1.pizzaIngredients,jt.pizza_ingred
```



```

from tab1 t1,
json_table(
concat('["',replace(t1.pizzaIngredients,',','"),"]'),
"$[*]" columns (pizza_ingred varchar(255) path "$")
) as jt),
tab3 as (
select * from tab2
)
select t3.pizza_ingred, count(*) as ingred_quantity
from tab3 t3
group by t3.pizza_ingred
order by ingred_quantity desc
limit 10;

```

	pizza_ingred	ingred_quantity
►	Garlic	27422
	Tomatoes	23694
	Red Onions	19547
	Red Peppers	16284
	Chicken	8443

14.

15. How many **repeat pizzas** appear across different orders?

```

select distinct pizza_name, count(distinct order_id) as number_of_order
from pizza_sales_replica
group by 1
order by 2 desc;

```

	pizza_name	number_of_order
►	The Classic Deluxe Pizza	2329
	The Hawaiian Pizza	2280
	The Pepperoni Pizza	2278
	The Barbecue Chicken Pizza	2273
	The Thai Chicken Pizza	2225
	The California Chicken Pizza	2197

16. Analyze customer preferences based on **pizza size and time of day**.

```
select day_part, pizza_size, count( distinct order_id) as num_of_orders
from pizza_sales_replica
group by 1,2
order by 1;
```

	day_part	pizza_size	num_of_orders
►	Afternoon	Large	5756
	Afternoon	Medium	5085
	Afternoon	Small	4828
	Afternoon	XLarge	243
	Afternoon	XXLarge	14

◆ Business KPIs

16. Calculate **Month-over-Month (MoM) growth**.

```
select month_number,
ROUND(((revenue - lag_month) / NULLIF(lag_month, 0)) * 100, 3)
as MOM_percentages
from(
select month_number,revenue,
lag(revenue) over(order by month_number asc) as lag_month
from(
select month_number, round(sum(total_price),0) as revenue
from(
select extract(month from order_Date) as month_number,
total_price
from pizza_sales_replica) base
group by 1) base_1) base_2;
```

	month_number	MOM_percentages
►	1	NULL
	2	-6.638
	3	8.037
	4	-2.358
	5	3.879

17. What is the **Revenue per Daypart** trend over time?

```
SELECT
  order_month,
  Day_part,
  SUM(total_price) AS total_rev
FROM pizza_sales_replica
GROUP BY order_month, Day_part
ORDER BY
  CASE order_month
    WHEN 'January' THEN 1
    WHEN 'February' THEN 2
    WHEN 'March' THEN 3
    WHEN 'April' THEN 4
    WHEN 'May' THEN 5
    WHEN 'June' THEN 6
    WHEN 'July' THEN 7
    WHEN 'August' THEN 8
    WHEN 'September' THEN 9
    WHEN 'October' THEN 10
    WHEN 'November' THEN 11
    WHEN 'December' THEN 12
  END,
  Day_part;
```

	order_month	Day_part	total_rev
▶	January	Afternoon	34741.2000000000026
	January	Evening	26481.5000000000003
	January	Morning	3507.900000000000005
	January	Night	5062.7
	February	Afternoon	31173.000000000000004

18. What are the **bottom 5 performing pizzas** and why?

```
select pizza_name, cast(sum(total_price) as decimal(10,0)) as total_rev
from pizza_sales_replica
group by 1
order by 2 asc
limit 5;
```

pizza_name	total_rev
The Brie Carre Pizza	11588
The Green Garden Pizza	13956
The Spinach Supreme Pizza	15278
The Mediterranean Pizza	15361
The Spinach Pesto Pizza	15596

19.

20. Build a **running total (cumulative sum)** of revenue by month.

```

SELECT
  order_year,
  order_month,
  SUM(total_price) OVER ( partition by order_year
    ORDER BY
      CASE order_month
        WHEN 'January' THEN 1
        WHEN 'February' THEN 2
        WHEN 'March' THEN 3
        WHEN 'April' THEN 4
        WHEN 'May' THEN 5
        WHEN 'June' THEN 6
        WHEN 'July' THEN 7
        WHEN 'August' THEN 8
        WHEN 'September' THEN 9
        WHEN 'October' THEN 10
        WHEN 'November' THEN 11
        WHEN 'December' THEN 12
      END ASC
    ) AS total_rev
FROM (
  SELECT
    order_year,
    order_month,
    SUM(total_price) AS total_price
  FROM pizza_sales_replica
  WHERE order_year = 2015
  GROUP BY order_year, order_month
) AS monthly_sales;

```

- - **alternative** - -

```

select distinct order_year, order_month, total_rev

```

```

from
(SELECT
  order_year,
  order_month,
  SUM(total_price) OVER (
    ORDER BY
    CASE order_month
      WHEN 'January' THEN 1
      WHEN 'February' THEN 2
      WHEN 'March' THEN 3
      WHEN 'April' THEN 4
      WHEN 'May' THEN 5
      WHEN 'June' THEN 6
      WHEN 'July' THEN 7
      WHEN 'August' THEN 8
      WHEN 'September' THEN 9
      WHEN 'October' THEN 10
      WHEN 'November' THEN 11
      WHEN 'December' THEN 12
    END ASC
  ) AS total_rev
FROM pizza_sales_replica) as tab1;

```

	order_year	order_month	total_rev
▶	2015	January	69793.29999999999
	2015	February	134952.89999999994
	2015	March	205350.00000000052
	2015	April	274086.80000000173
	2015	May	345489.55000000406

♦ Advanced SQL Tasks

21. Create a **cohort** of pizzas launched in a specific quarter and track performance.
22. Use **window functions** to rank pizzas by sales for all months together.

```

select dense_rank() over( order by revenue desc) as Rank_by_revenue, Best_Seller,
revenue
from(
select distinct pizza_name as best_Seller,round(sum(total_price),0) as revenue
from pizza_sales_replica

```

group by 1) as t;

```
select dense_rank() over( order by revenue desc) as Rank_by_revenue, Best_Seller, revenue
from(
select distinct pizza_name as best_Seller,round(sum(total_price),0) as revenue
from pizza_sales_replica
group by 1) as t;
```

23. Find top 3 pizzas_name by sales for each month.

```
Select Rank_by_revenue,order_month, Best_Seller, revenue
from(
select dense_rank() over( partition by month_number order by revenue desc) as
Rank_by_revenue,month_number,order_month, Best_Seller, revenue
from(
select pizza_name as best_Seller,EXTRACT(MONTH FROM order_date) as
month_number,order_month ,round(sum(total_price),0) as revenue
from pizza_sales_replica
group by 1,2,3) as t) as t1
where Rank_by_revenue in (1,2,3);
```

	Rank_by_revenue	order_month	Best_Seller	revenue
▶	1	January	The Barbecue Chicken Pizza	3770
	2	January	The Thai Chicken Pizza	3657
	3	January	The California Chicken Pizza	3472
	1	February	The California Chicken Pizza	3537
	2	February	The Barbecue Chicken Pizza	3417

24. Create **dynamic grouping** (e.g., top 3 vs others).

```
-- using ntile()

select ntile(11) over(order by revenue desc) as top_To_bottom,pizza_name,revenue
from(
select pizza_name, sum(total_price) as revenue
from pizza_sales_replica
group by 1) t;
```

	top_To_bottom	pizza_name	revenue
▶	1	The Thai Chicken Pizza	43434.25
	1	The Barbecue Chicken Pizza	42768
	1	The California Chicken Pizza	41409.5
	2	The Classic Deluxe Pizza	38180.5
	2	The Spicy Italian Pizza	34831.25

```
-- using dense_rank()
select revenue,
case
    when top_rank<=3 then pizza_name
    else NULL
end as top_3_pizza,
case
    when top_rank>3 then pizza_name
    end as other_pizzas
from(
select dense_rank() over( order by revenue desc) as top_rank,pizza_name,revenue
from(
select pizza_name, sum(total_price) as revenue
from pizza_sales_replica
group by 1) t) as t2;
```

	revenue	top_3_pizza	other_pizzas
▶	43434.25	The Thai Chicken Pizza	NULL
	42768	The Barbecue Chicken Pizza	NULL
	41409.5	The California Chicken Pizza	NULL
	38180.5	NULL	The Classic Deluxe Pizza
	34831.25	NULL	The Spicy Italian Pizza

25. Build a **pizza sales funnel** (ordered → repeat ordered → top-rated)

```
select pizza_name, ord_number, total_rev,
case
    when ord_number>=2000 then 'Top_seller'
    when ord_number between 1000 and 2000 then 'Mid_seller'
    else 'low_seller'
end as based_on_number_of_sell
from
```

```

(with tab as (
select pizza_name, count(order_id) as ord_number
from(
select order_id, pizza_name
from pizza_sales_replica
) t
group by 1
having count(order_id)>=1
),
tab1 as (
select pizza_name,sum( total_price) as total_rev
from pizza_sales_replica
group by 1
)
select tab1.pizza_name,tab.ord_number, tab1.total_rev
from tab1
join tab on tab.pizza_name=tab1.pizza_name
order by tab.ord_number desc) final;

```

	pizza_name	ord_number	total_rev	based_on_number_of_sell
►	The Classic Deluxe Pizza	2416	38180.5	Top_seller
	The Barbecue Chicken Pizza	2372	42768	Top_seller
	The Hawaiian Pizza	2370	32273.25	Top_seller
	The Pepperoni Pizza	2369	30161.75	Top_seller
	The Thai Chicken Pizza	2315	43434.25	Top_seller

26. Analyze **average revenue per pizza over time**.

```

select order_month, pizza_name,count(order_id) as number_of_order,sum(total_price) as
totat_revenue,
avg(total_price) as average_revenue
from pizza_sales_replica
group by 1,2;

```

	order_month	pizza_name	number_of_order	totat_revenue	average_revenue
►	January	The Hawaiian Pizza	182	2442.75	13.421703296703297
	January	The Classic Deluxe Pizza	189	2941.5	15.563492063492063
	January	The Five Cheese Pizza	134	2553	19.05223880597015
	January	The Italian Supreme Pizza	166	2954.75	17.799698795180724
	January	The Mexicana Pizza	129	2358.25	18.281007751937985